Linear Quadratic Model Predictive Control: Notes from Theory to Praxis

Timothy Werner

Abstract. This is LQMPC – Linear-quadratic Model Predictive Control
This paper is still work in progress.

Keywords. MPC, IPM, Riccati

Contents

1 Introduction **1**

2 The linear-quadratic OCP **2**

3 Unconstrained linear-quadratic OCP **3**
3.1 Solution via Equality Constraint Elimination 3
3.2 Solution via Karush–Kuhn–Tucker 4
3.2.1 From KKT to Block-Cholesky 6
3.2.2 From KKT to Dynamic Programming 7
3.3 The Riccati Recursion 11

References **15**

1 Introduction

2 The linear-quadratic OCP

The finite-horizon linear-quadratic optimal control problem is formulated as follows.

$$
\begin{aligned}
\underset{u_k, x_k}{\text{minimize}} \quad & \sum_{k=0}^{N} \frac{1}{2} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} R_k & S_k & r_k \\ S_k^\top & Q_k & q_k \\ r_k^\top & q_k^\top & 0 \end{bmatrix} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix} \\
\text{subject to} \quad & x_0 = \hat{x} \\
& x_{k+1} = A_k x_k + B_k u_k + b_k && , k \in \mathcal{I}_0^{N-1} \\
& \underline{x}_k \leq J_k^{bx} x_k \leq \bar{x}_k && , k \in \mathcal{I}_0^{N} \\
& \underline{u}_k \leq J_k^{bu} u_k \leq \bar{u}_k && , k \in \mathcal{I}_0^{N-1} \\
& D_k u_k + C_k x_k \leq d_k && , k \in \mathcal{I}_0^{N}
\end{aligned}
$$

With the states $x_k \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ and inputs $u_k \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$. The state and input dimensions are n_x and n_u, respectively. The time horizon is $N \in \mathbb{N}_{>0}$ and the initial state is given as $\hat{x} \in \mathcal{X}$. The system dynamics are described by the matrices $A_k \in \mathbb{R}^{n_x \times n_x}$, $B_k \in \mathbb{R}^{n_x \times n_u}$ and the vector $b_k \in \mathbb{R}^{n_x}$. The cost function is defined by the input weight matrices $R_k \in \mathbb{S}_{++}^{n_u}$, the state weight matrices $Q_k \in \mathbb{S}_{++}^{n_x}$ and the cross-term weight matrices $S_k \in \mathbb{R}^{n_u \times n_x}$, which are collected in the stage Hessian

$$
H_k = \begin{bmatrix} R_k & S_k \\ S_k^\top & Q_k \end{bmatrix}.
$$

The linear terms of the cost function are given by the vectors $r_k \in \mathbb{R}^{n_u}$ and $q_k \in \mathbb{R}^{n_x}$. We assume $H_k \succ 0$ for $k \in \mathcal{I}_0^{N-1}$ (equivalently $R_k \succ 0$ and $Q_k - S_k^\top R_k^{-1} S_k \succ 0$), as well as $Q_N \succ 0$. This single premise makes the problem strictly convex, so its solution is unique, and it is exactly what the factorization-based methods derived in this paper require. The terminal stage $k = N$ carries no input, i.e. $n_{u,N} = 0$, so that only Q_N, q_N and the state constraints appear at this stage (D_N is empty), while the dynamics and the box constraints on the inputs end at $N - 1$. There are a number of n_{bx} box constraints on the states and n_{bu} box constraints on the inputs, which are mapped by the selection matrices $J_k^{bx} \in \mathbb{R}^{n_{bx} \times n_x}$ and $J_k^{bu} \in \mathbb{R}^{n_{bu} \times n_u}$, together with the lower bounds $\underline{x}_k \in \mathbb{R}^{n_{bx}}$, $\underline{u}_k \in \mathbb{R}^{n_{bu}}$ and upper bounds $\bar{x}_k \in \mathbb{R}^{n_{bx}}$, $\bar{u}_k \in \mathbb{R}^{n_{bu}}$ respectively. There are a number of n_p polytopic constraints represented by the matrices $D_k \in \mathbb{R}^{n_p \times n_u}$, $C_k \in \mathbb{R}^{n_p \times n_x}$ and the vector $d_k \in \mathbb{R}^{n_p}$. The integer set is denoted as $\mathcal{I}_a^b := \{a, a+1, \dots b\}$. The OCP notation is largely borrowed from HPIPM [7], and most of the derivations in this paper follow those presented there.

3 Unconstrained linear-quadratic OCP

This section serves to explain the solution of the KKT system, which sits at the core of solving linear-quadratic optimal control problems. We will first consider the unconstrained case to gain a better understanding of the KKT system and its importance to the problem. The unconstrained finite-horizon linear-quadratic optimal control problem is formulated as follows.

$$\begin{aligned} \underset{u_k, x_k}{\text{minimize}} \quad & \sum_{k=0}^{N} \frac{1}{2} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} R_k & S_k & r_k \\ S_k^\top & Q_k & q_k \\ r_k^\top & q_k^\top & 0 \end{bmatrix} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix} \\ \text{subject to} \quad & x_0 = \hat{x} \\ & x_{k+1} = A_k x_k + B_k u_k + b_k, \quad k \in \mathcal{I}_0^{N-1} \end{aligned}$$

Note that *unconstrained* in this case means that there are no *inequality* constraints on states or inputs. The only constraints are the system dynamics and the initial state. They do restrict the feasible set, namely to an affine subspace in which the states are determined by the inputs and the initial state. Unlike inequality constraints, however, they introduce no combinatorial structure (no active set), so the optimality conditions remain a linear system. We now look at how to compute a solution to this problem.

3.1. Solution via Equality Constraint Elimination. The optimization problem is formulated such that the decision variables are the input and state pairs (u_k, x_k) for all stages k over the time horizon N. Consecutive stages k and $k+1$ are coupled via the system dynamics in the form of the equality constraints

$$x_{k+1} = A_k x_k + B_k u_k + b_k, \, \forall k \in \mathcal{I}_0^{N-1}$$

This means that the state of a future stage x_{k+1} is completely determined by the state of the current stage x_k and the input of the current stage u_k. This iteration starts from the initial condition $x_0 = \hat{x}$ at the beginning of the time horizon. This allows us to eliminate the states from the decision variables, leaving only the inputs as decision variables, parametrized by the initial condition $\hat{x}$. To see this, let's explicitly calculate the state trajectory, starting at the initial stage.

$$x_0 = \hat{x}$$

The first stage is given by the first equality constraint

$$x_1 = A_0 \hat{x} + B_0 u_0 + b_0$$

The second stage is given by the second equality constraint

$$x_2 = A_1 x_1 + B_1 u_1 + b_1$$

We can now substitute the first stage into the second stage, eliminating x_1

$$\begin{aligned} x_2 &= A_1 x_1 + B_1 u_1 + b_1 \\ x_2 &= A_1 \left(A_0 \hat{x} + B_0 u_0 + b_0 \right) + B_1 u_1 + b_1 \\ x_2 &= A_1 A_0 \hat{x} + A_1 B_0 u_0 + B_1 u_1 + A_1 b_0 + b_1 \end{aligned}$$

We can do this for each stage and thus eliminate all states x_k. The iteration can be written in closed form for any stage $j \in \mathcal{I}_1^N$ as

$$x_j = \prod_{k=0}^{j-1} A_k \hat{x} + \sum_{k=0}^{j-1} \prod_{i=k+1}^{j-1} A_i B_k u_k + \sum_{k=0}^{j-1} \prod_{i=k+1}^{j-1} A_i b_k$$

Here the products are ordered with decreasing index, $\prod_{i=k+1}^{j-1} A_i := A_{j-1} \cdots A_{k+1}$, and an empty product is the identity. In matrix-vector form, the state trajectory reads

$$\mathbf{x} = \Psi\, \hat{x} + \Gamma\, \mathbf{u} + \Theta\, \mathbf{b}$$

where $\mathbf{x} = [x_1; \ldots; x_N]$, $\mathbf{u} = [u_0; \ldots; u_{N-1}]$, $\mathbf{b} = [b_0; \ldots; b_{N-1}]$ and

$$\Psi = \begin{bmatrix} A_0 \\ A_1 A_0 \\ \vdots \\ \prod_{k=0}^{N-1} A_k \end{bmatrix}$$

$$\Gamma = \begin{bmatrix} B_0 & 0 & \cdots & 0 \\ A_1 B_0 & B_1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \prod_{i=1}^{N-1} A_i\, B_0 & \cdots & A_{N-1} B_{N-2} & B_{N-1} \end{bmatrix}$$

$$\Theta = \begin{bmatrix} I & 0 & \cdots & 0 \\ A_1 & I & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \prod_{i=1}^{N-1} A_i & \cdots & A_{N-1} & I \end{bmatrix}$$

This expression can be substituted back into the objective, leaving a condensed QP in the inputs u_k only, parametrized by the initial condition $\hat{x}$. Since the dimension of the problem has decreased from roughly $N(n_x + n_u)$ to Nn_u, this process is known as *condensing* [9]. The benefit of a smaller dimension comes at the cost of a dense Hessian. To state the Hessian, we stack the stage weights as $\mathbf{R} := \mathrm{blkdiag}(R_0, \ldots, R_{N-1})$ and $\mathbf{Q} := \mathrm{blkdiag}(Q_1, \ldots, Q_N)$, matching the stacked states $x_1, \ldots, x_N$. The initial state $x_0 = \hat{x}$ is fixed and thus contributes to the parametric part of the problem only. The same holds for the cross-term $u_0^\top S_0 \hat{x}$ of the first stage. The remaining cross-terms $u_k^\top S_k x_k$, $k \in \mathcal{I}_1^{N-1}$, pair the input u_k with the state x_k and are collected in the block matrix $\bar{\mathbf{S}} \in \mathbb{R}^{Nn_u \times Nn_x}$, which has the blocks $S_1, \ldots, S_{N-1}$ on its first block sub-diagonal and is zero elsewhere. Note that the pairing is shifted by one index, because x_k is the k-th block of $\mathbf{x}$, while u_k is the $(k+1)$-th block of $\mathbf{u}$. With this, the Hessian after the condensing is given as

$$\mathcal{H}_c = \mathbf{R} + \Gamma^\top \mathbf{Q} \Gamma + \bar{\mathbf{S}} \Gamma + \Gamma^\top \bar{\mathbf{S}}^\top$$

For the special case of no cross-terms, $S_k = 0$, this reduces to $\mathbf{R} + \Gamma^\top \mathbf{Q} \Gamma$. Since this paper focuses on sparsity-exploiting methods, we do not pursue condensing further. For an in-depth discussion and implementation of this method, have a look at qpOASES [5].

3.2. Solution via Karush–Kuhn–Tucker. In terms of numerical optimization, the OCP above is an equality-constrained QP, which is strictly convex in the decision variables u_k and x_k. This holds because the cost function is quadratic in u_k and x_k with $H_k \succ 0$ for $k < N$ and $Q_N \succ 0$ by assumption and the equality constraints are affine in u_k and x_k. Strict convexity guarantees that the solution is unique. This solution is fully characterized by the Karush–Kuhn–Tucker (KKT) conditions, which for a convex QP with affine equality constraints are necessary and sufficient. To derive the KKT conditions, we first form the Lagrangian by introducing the Lagrange multipliers $\pi_0 \in \mathbb{R}^{n_x}$ for the initial condition and $\pi_{k+1} \in \mathbb{R}^{n_x}$ for the dynamic constraint of stage k.

$$\mathcal{L} = \sum_{k=0}^{N-1} l_k(u_k, x_k) + l_N(x_N) + \pi_0^\top (\hat{x} - x_0) + \sum_{k=0}^{N-1} \pi_{k+1}^\top (A_k x_k + B_k u_k + b_k - x_{k+1})$$

With the stage cost

$$l_k(u_k, x_k) = \frac{1}{2}u_k^\top R_k u_k + \frac{1}{2}x_k^\top Q_k x_k + u_k^\top S_k x_k + r_k^\top u_k + q_k^\top x_k$$

and terminal cost

$$l_N(x_N) = \frac{1}{2}x_N^\top Q_N x_N + q_N^\top x_N$$

The KKT conditions are the stationarity conditions, which state that the Lagrangian is stationary at the optimal solution (u_k^*, x_k^*, π_k^*) with respect to all primal and dual variables.

$$\nabla\mathcal{L}(u_k^*, x_k^*, \pi_k^*) = 0$$

The stationarity conditions are given as follows. Stationarity w.r.t. inputs u_k for $k \in \mathcal{I}_0^{N-1}$:

$$\nabla_{u_k}\mathcal{L} = R_k u_k + S_k x_k + r_k + B_k^\top \pi_{k+1} = 0$$

Stationarity w.r.t. states x_k for $k \in \mathcal{I}_0^{N-1}$:

$$\nabla_{x_k}\mathcal{L} = Q_k x_k + S_k^\top u_k + q_k + A_k^\top \pi_{k+1} - \pi_k = 0$$

and for the terminal state:

$$\nabla_{x_N}\mathcal{L} = Q_N x_N + q_N - \pi_N = 0$$

Stationarity w.r.t. Lagrange multipliers π_k, which recovers the initial condition and the dynamics:

$$\nabla_{\pi_0}\mathcal{L} = \hat{x} - x_0 = 0, \qquad \nabla_{\pi_{k+1}}\mathcal{L} = A_k x_k + B_k u_k + b_k - x_{k+1} = 0, \quad k \in \mathcal{I}_0^{N-1}$$

Introducing π_0 makes every state condition read uniformly with $-\pi_k$. If we stack the decision variables and Lagrange multipliers into a single vector, we can write the KKT conditions as a large linear system. This system is called the KKT system. Here is an example showing the structure for $N = 2$, with the variable ordering

$$z = [u_0;\, x_0;\, u_1;\, x_1;\, x_2;\, \pi_0;\, \pi_1;\, \pi_2].$$

$$\begin{bmatrix}
R_0 & S_0 & & & & & B_0^\top & \\
S_0^\top & Q_0 & & & & -I & A_0^\top & \\
 & & R_1 & S_1 & & & & B_1^\top \\
 & & S_1^\top & Q_1 & & & -I & A_1^\top \\
 & & & & Q_2 & & & -I \\
 & -I & & & & & & \\
B_0 & A_0 & & -I & & & & \\
 & & B_1 & A_1 & -I & & &
\end{bmatrix}
\begin{bmatrix} u_0 \\ x_0 \\ u_1 \\ x_1 \\ x_2 \\ \pi_0 \\ \pi_1 \\ \pi_2 \end{bmatrix}
= -
\begin{bmatrix} r_0 \\ q_0 \\ r_1 \\ q_1 \\ q_2 \\ \hat{x} \\ b_0 \\ b_1 \end{bmatrix}$$

The system can be summarized in the following form, which is the standard system arising from optimal control problems.

$$\begin{bmatrix} \mathcal{H} & \mathcal{A}^\top \\ \mathcal{A} & 0 \end{bmatrix}
\begin{bmatrix} y \\ \pi \end{bmatrix}
= -
\begin{bmatrix} h \\ c \end{bmatrix}$$

with the primal vector $y = [u_0; x_0; \ldots; u_{N-1}; x_{N-1}; x_N]$, the multipliers $\pi = [\pi_0; \ldots; \pi_N]$, the gradient $h = [r_0; q_0; \ldots; r_{N-1}; q_{N-1}; q_N]$ and the right-hand side $c = [\hat{x}; b_0; \ldots; b_{N-1}]$. Looking closely at this system, we can see that the Hessian $\mathcal{H}$ is block-diagonal with $\mathcal{H} = \text{blkdiag}(H_0, \ldots, H_{N-1}, Q_N)$ and $\mathcal{A}$ is block-bidiagonal. That means the KKT system obeys a certain sparsity pattern. This pattern can be exploited for fast and reliable solutions of the system, which is the core idea of tailored LQMPC solvers [7, 14] and is

the main focus of this paper. There are in general three approaches to solve this system. The first and naive approach would be to feed this system to a dense symmetric-indefinite factorization solver. This works, but finding the solution is of order $\mathcal{O}\big(N^3(n_x+n_u)^3\big)$, i.e. cubic in the horizon, which is impractical for all but short horizons. The other two approaches are worth considering in a bit more detail.

3.2.1. From KKT to Block-Cholesky. The second approach is to notice the *spatial* sparsity pattern of the Hessian $\mathcal{H}$ and realize that it carries over to the normal equations, which have a block-tridiagonal structure. To show this, let us attempt to solve this system by eliminating y from the KKT system, via the Schur complement. We thus form

$$y = \mathcal{H}^{-1}(-h - \mathcal{A}^\top \pi)$$

and substitute into $\mathcal{A}y = -c$, from which we get

$$\mathcal{A}\mathcal{H}^{-1}\mathcal{A}^\top \pi = c - \mathcal{A}\mathcal{H}^{-1}h$$

We now define $Y := \mathcal{A}\mathcal{H}^{-1}\mathcal{A}^\top$ and $\beta := c - \mathcal{A}\mathcal{H}^{-1}h$, to arrive at the system in the so-called normal equations form [14].

$$Y\pi = \beta$$

We now examine the spatial structure of Y more closely. Here we need $\mathcal{H} \succ 0$, i.e. $H_k \succ 0$ for all stages including $Q_N \succ 0$, which holds by assumption. Then every H_k is invertible and, since $\mathcal{A}$ has full row rank, $Y \succ 0$ admits a Cholesky factorization. We see that the matrix $\mathcal{H}$ is block-diagonal with the stage Hessian blocks

$$H_k = \begin{bmatrix} R_k & S_k \\ S_k^\top & Q_k \end{bmatrix}$$

and the matrix $\mathcal{A}$ is block-bidiagonal. With the stacked dynamics $G_k := \begin{bmatrix} B_k & A_k \end{bmatrix}$ and the state selector $E_k := \begin{bmatrix} 0 & I_{n_x} \end{bmatrix}$, such that $x_k = E_k y_k$ for the stage variable $y_k = [u_k; x_k]$, the block rows of $\mathcal{A}$ read $-E_0 y_0$ and $G_k y_k - E_{k+1} y_{k+1}$. At the terminal stage there is no input, so $y_N = x_N$, $E_N = I_{n_x}$ and $H_N = Q_N$. Thus Y has a block-tridiagonal structure in the following way.

$$Y = \begin{bmatrix} Y_{00} & Y_{01} & & \\ Y_{01}^\top & Y_{11} & Y_{12} & \\ & Y_{12}^\top & Y_{22} & \ddots \\ & & \ddots & \ddots \end{bmatrix}$$

With $\Phi_k := H_k^{-1}$, the blocks of Y are given as

$$Y_{00} = E_0 \Phi_0 E_0^\top, \qquad Y_{k,k} = G_{k-1}\Phi_{k-1}G_{k-1}^\top + E_k \Phi_k E_k^\top \quad (k \geq 1)$$

$$Y_{k,k+1} = -E_k \Phi_k G_k^\top$$

The next step is the Cholesky factorization of Y

$$Y = L_Y L_Y^\top$$

Due to the block-tridiagonal structure of Y, L_Y is block-lower-bidiagonal:

$$L_Y = \begin{bmatrix} L_{00}^Y & & & \\ L_{10}^Y & L_{11}^Y & & \\ & L_{21}^Y & L_{22}^Y & \\ & & \ddots & \ddots \end{bmatrix}$$

This fact motivates a blockwise factorization with the following recursion for $k = 0, \ldots, N$, where the subtracted term is absent for $k = 0$ and the second equation applies only for $k < N$

$$L^Y_{k,k} {L^Y_{k,k}}^\top = Y_{k,k} - L^Y_{k,k-1} {L^Y_{k,k-1}}^\top$$

$$L^Y_{k+1,k} {L^Y_{k,k}}^\top = Y_{k+1,k}, \qquad Y_{k+1,k} = Y_{k,k+1}^\top$$

Finally, solve $L_Y v^Y = \beta$ for the intermediate vector v^Y by forward substitution, solve $L_Y^\top \pi = v^Y$ by backward substitution, and recover $y = \mathcal{H}^{-1}(-h - \mathcal{A}^\top \pi)$. For an in-depth discussion and implementation of this method, have a look at FORCES [4].

3.2.2. From KKT to Dynamic Programming. The third approach is to notice that the problem also has temporal structure. Strictly speaking, the spatial structure exploited above also stems from a temporal structure, but the interpretation takes a different route. A closer look at the KKT conditions reveals the temporal relationship. The stationarity conditions with respect to x_k can be rewritten as

$$\pi_k = Q_k x_k + S_k^\top u_k + q_k + A_k^\top \pi_{k+1}$$

This reveals that the dynamics of the costates π_k propagate *backward* in time. They are fixed by the terminal condition $\pi_N = Q_N x_N + q_N$ at the end of the time horizon. The stationarity conditions with respect to π_k reveal that the dynamics of the states x_k propagate *forward* in time.

$$x_{k+1} = A_k x_k + B_k u_k + b_k$$

They are fixed by the initial condition $x_0 = \hat{x}$ at the beginning of the time horizon. Both dynamics are coupled via the stationarity conditions for the inputs u_k

$$R_k u_k + S_k x_k + r_k + B_k^\top \pi_{k+1} = 0$$

These equations form a coupled two-point boundary value problem (BVP) [3]. They are not an initial value problem, which can be solved by forward integration. Nor are they a final value problem, which can be solved by backward integration. Consider solving it forward in time. The iteration starts at stage $k = 0$ with $x_0 = \hat{x}$. Computing u_0^* requires π_1. But π_1 depends on π_2, which depends on π_3, all the way to π_N. And π_N depends on x_N, which is unknown before the forward simulation is complete. Solving it backward fails for the same reason. The iteration would start at stage $k = N$ with $\pi_N = Q_N x_N + q_N$, but x_N is not known without simulating forward from x_0. The information flows in opposite directions, and the two directions are coupled through the optimality condition for u_k. This is the essence of the two-point BVP. Data is specified at both boundaries of the time horizon, so the equations cannot simply be iterated in one direction. The block-Cholesky approach above resolves this coupling by factoring the whole system at once. Dynamic programming, based on Bellman's principle of optimality [1], resolves it differently, and it is a fundamental shift of perspective. Instead of searching for trajectories x^*, u^* and π^* that are characterized by the KKT conditions, one searches for an optimal policy, i.e. a function $u_k^*(x_k)$ of the state x_k. This decouples the two-point BVP and allows solving it in one backward pass (policy computation) and one forward pass (trajectory rollout).

Bellman's principle of optimality. Consider the total cost-to-go from stage k to the terminal stage N

$$J_k(x_k, u_k, \ldots, u_{N-1}) = \sum_{j=k}^{N-1} l_j(u_j, x_j) + l_N(x_N)$$

subject to the dynamics $x_{j+1} = A_j x_j + B_j u_j + b_j$. The *value function* $V_k(x_k)$ is defined as the minimum of this cost-to-go over all admissible input sequences

$$V_k(x_k) = \min_{u_k,\dots,u_{N-1}} J_k(x_k, u_k, \dots, u_{N-1})$$

Bellman's principle of optimality states: *An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.* [1, p. 83] Applied to our problem, this yields the recursive relationship known as the *Bellman equation* [2]

$$V_k(x_k) = \min_{u_k} \{l_k(u_k, x_k) + V_{k+1}(A_k x_k + B_k u_k + b_k)\}$$

with the terminal condition

$$V_N(x_N) = l_N(x_N) = \frac{1}{2} x_N^\top Q_N x_N + q_N^\top x_N$$

The Bellman equation is the foundation of the backward recursion. At each stage k, the minimization is only over the *current* input u_k. All future costs are already captured in the value function V_{k+1}. This is how dynamic programming resolves the coupling of the two-point BVP: x_N never needs to be known explicitly. Only V_{k+1} is needed as a function of the state, and the dynamics do the rest.

Quadratic structure of the value function. Since the terminal cost is quadratic in x_N, and each stage adds quadratic terms and propagates through affine dynamics, we expect the value function to remain quadratic at every stage [2]. We thus make the ansatz

$$V_k(x_k) = \frac{1}{2} x_k^\top P_k x_k + p_k^\top x_k + \eta_k$$

with $P_k \in \mathbb{S}_+^{n_x}$, $p_k \in \mathbb{R}^{n_x}$ and $\eta_k \in \mathbb{R}$. This ansatz will be verified by showing that if it holds at stage $k+1$, it also holds at stage k. The terminal condition gives

$$P_N = Q_N, \qquad p_N = q_N, \qquad \eta_N = 0$$

Expanding the Bellman equation. The expression minimized in the Bellman equation is the *action-value function* (or $\mathcal{Q}$-function) [15]

$$\mathcal{Q}_k(u_k, x_k) := l_k(u_k, x_k) + V_{k+1}(A_k x_k + B_k u_k + b_k)$$

such that $V_k(x_k) = \min_{u_k} \mathcal{Q}_k(u_k, x_k)$. Substituting the ansatz for V_{k+1}, we expand it term by term. The stage cost contributes

$$l_k(u_k, x_k) = \frac{1}{2} u_k^\top R_k u_k + \frac{1}{2} x_k^\top Q_k x_k + u_k^\top S_k x_k + r_k^\top u_k + q_k^\top x_k$$

For the value function term, denote $\xi := A_k x_k + B_k u_k + b_k$. Then

$$V_{k+1}(\xi) = \frac{1}{2} \xi^\top P_{k+1} \xi + p_{k+1}^\top \xi + \eta_{k+1}$$

Expanding $\xi^\top P_{k+1} \xi$ gives

$$\begin{aligned}
\xi^\top P_{k+1} \xi &= (A_k x_k + B_k u_k + b_k)^\top P_{k+1} (A_k x_k + B_k u_k + b_k) \\
&= x_k^\top A_k^\top P_{k+1} A_k x_k + u_k^\top B_k^\top P_{k+1} B_k u_k + b_k^\top P_{k+1} b_k \\
&\quad + 2\, u_k^\top B_k^\top P_{k+1} A_k x_k + 2\, b_k^\top P_{k+1} A_k x_k + 2\, b_k^\top P_{k+1} B_k u_k
\end{aligned}$$

and $p_{k+1}^\top \xi = p_{k+1}^\top A_k x_k + p_{k+1}^\top B_k u_k + p_{k+1}^\top b_k$. Now collect all terms in $\mathcal{Q}_k(u_k, x_k)$ according to their dependence on x_k and u_k. The result is again a quadratic form in (u_k, x_k)

$$\mathcal{Q}_k(u_k, x_k) = \frac{1}{2} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} \tilde{R}_k & \tilde{S}_k & \tilde{r}_k \\ \tilde{S}_k^\top & \tilde{Q}_k & \tilde{q}_k \\ \tilde{r}_k^\top & \tilde{q}_k^\top & 2\tilde{\eta}_k \end{bmatrix} \begin{bmatrix} u_k \\ x_k \\ 1 \end{bmatrix}$$

with the *augmented stage cost* terms

$$\tilde{Q}_k = Q_k + A_k^\top P_{k+1} A_k \tag{1}$$
$$\tilde{S}_k = S_k + B_k^\top P_{k+1} A_k \tag{2}$$
$$\tilde{R}_k = R_k + B_k^\top P_{k+1} B_k \tag{3}$$
$$\tilde{q}_k = q_k + A_k^\top (P_{k+1} b_k + p_{k+1}) \tag{4}$$
$$\tilde{r}_k = r_k + B_k^\top (P_{k+1} b_k + p_{k+1}) \tag{5}$$
$$\tilde{\eta}_k = \tfrac{1}{2} b_k^\top P_{k+1} b_k + p_{k+1}^\top b_k + \eta_{k+1} \tag{6}$$

Note the structure: these are the original stage cost matrices, augmented by the contribution of the future cost V_{k+1} propagated backward through the dynamics. The matrix $\tilde{Q}_k$ is the state cost after absorbing the future cost seen through A_k, $\tilde{R}_k$ is the input cost after absorbing the future cost seen through B_k, and $\tilde{S}_k$ captures the cross-coupling.

Minimization: the optimal policy. Assume $P_{k+1} \succeq 0$, which holds for $k+1 = N$. Then $\tilde{R}_k = R_k + B_k^\top P_{k+1} B_k$ inherits positive definiteness from $R_k \succ 0$, so the $\mathcal{Q}$-function is strictly convex in u_k and the minimum is unique. Moreover, the Hessian of the $\mathcal{Q}$-function in (u_k, x_k) is the *augmented stage Hessian* $\tilde{H}_k := H_k + G_k^\top P_{k+1} G_k \succeq H_k \succ 0$, with $G_k = \begin{bmatrix} B_k & A_k \end{bmatrix}$, and P_k is its Schur complement with respect to $\tilde{R}_k$, see below. Hence $P_k \succ 0$ by induction from $P_N = Q_N \succ 0$, so this holds at every stage. Setting $\nabla_{u_k} \mathcal{Q}_k = 0$ yields

$$\tilde{R}_k u_k + \tilde{S}_k x_k + \tilde{r}_k = 0$$

Solving for the optimal input gives

$$u_k^* = -\tilde{R}_k^{-1} \left(\tilde{S}_k x_k + \tilde{r}_k \right) = K_k x_k + \kappa_k \tag{7}$$

where we define the *feedback gain* and *feedforward term*

$$K_k := -\tilde{R}_k^{-1} \tilde{S}_k \tag{8}$$
$$\kappa_k := -\tilde{R}_k^{-1} \tilde{r}_k \tag{9}$$

This is the optimal policy we were looking for: an affine state-feedback law, parametrized at each stage by the gain K_k and the offset κ_k. The gain K_k depends only on the dynamics, the quadratic weights and P_{k+1}, not on the linear terms or the offsets b_k: it is the time-varying *LQR gain* [11]. The feedforward term κ_k collects the affine terms in the dynamics and the linear terms in the cost, and it vanishes when q_k, r_k, b_k and q_N are all zero.

The Riccati recursion. Substituting the optimal policy (7) back into the $\mathcal{Q}$-function and collecting terms in x_k, we verify the quadratic ansatz and obtain the recursion for P_k and p_k. The value function at stage k is

$$V_k(x_k) = \mathcal{Q}_k(u_k^*, x_k) = \frac{1}{2} x_k^\top P_k x_k + p_k^\top x_k + \eta_k$$

To obtain P_k, collect all terms quadratic in x_k from $\mathcal{Q}_k(K_k x_k + \kappa_k, x_k)$. The quadratic contribution from $u_k^* = K_k x_k + \kappa_k$ is

$$\frac{1}{2}x_k^\top \left(\tilde{Q}_k + K_k^\top \tilde{S}_k + \tilde{S}_k^\top K_k + K_k^\top \tilde{R}_k K_k\right) x_k$$

Substituting $K_k = -\tilde{R}_k^{-1}\tilde{S}_k$ and simplifying (using $K_k^\top \tilde{R}_k K_k = \tilde{S}_k^\top \tilde{R}_k^{-1}\tilde{S}_k$ and $K_k^\top \tilde{S}_k + \tilde{S}_k^\top K_k = -2\tilde{S}_k^\top \tilde{R}_k^{-1}\tilde{S}_k$) yields

$$P_k = \tilde{Q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1}\tilde{S}_k \tag{10}$$

This is the *Riccati difference equation*. For time-invariant data, its stationary solution satisfies the discrete algebraic Riccati equation (DARE) of the infinite-horizon LQR problem [2]. Expanding the shorthand (1)–(3), the fully explicit form reads

$$P_k = Q_k + A_k^\top P_{k+1} A_k - (S_k + B_k^\top P_{k+1} A_k)^\top (R_k + B_k^\top P_{k+1} B_k)^{-1}(S_k + B_k^\top P_{k+1} A_k)$$

Similarly, collecting all terms linear in x_k gives

$$p_k = \tilde{q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1}\tilde{r}_k \tag{11}$$

and the constant term (which is not needed for computing the optimal trajectory, but completes the recursion) is

$$\eta_k = \tilde{\eta}_k - \frac{1}{2}\tilde{r}_k^\top \tilde{R}_k^{-1}\tilde{r}_k$$

Equations (10) and (11) together with the definitions (1)–(5) and the terminal conditions $P_N = Q_N$, $p_N = q_N$ form the *backward Riccati recursion*. The constant η_k, via (6) and $\eta_N = 0$, is only needed if the optimal cost itself is of interest. Note the Schur complement structure: P_k is obtained from the augmented stage Hessian $\tilde{H}_k$ by eliminating the input block via the Schur complement with respect to $\tilde{R}_k$. This is not a coincidence. It is the matrix-level manifestation of the minimization over u_k.

Connection to the costates. The value function gradient with respect to the state is

$$\nabla_{x_k} V_k(x_k) = P_k x_k + p_k$$

We now show that this equals the costate π_k from the KKT conditions. Recall the costate dynamics

$$\pi_k = Q_k x_k + S_k^\top u_k + q_k + A_k^\top \pi_{k+1}$$

If we define $\pi_k := P_k x_k + p_k$ and substitute $u_k^* = K_k x_k + \kappa_k$ and $\pi_{k+1} = P_{k+1}x_{k+1} + p_{k+1}$ with $x_{k+1} = A_k x_k + B_k u_k^* + b_k$, the right-hand side becomes

$$\begin{aligned}
&Q_k x_k + S_k^\top (K_k x_k + \kappa_k) + q_k + A_k^\top \left[P_{k+1}(A_k x_k + B_k(K_k x_k + \kappa_k) + b_k) + p_{k+1}\right] \\
&= \left[Q_k + S_k^\top K_k + A_k^\top P_{k+1}(A_k + B_k K_k)\right] x_k \\
&\quad + \left[S_k^\top \kappa_k + q_k + A_k^\top P_{k+1}(B_k \kappa_k + b_k) + A_k^\top p_{k+1}\right]
\end{aligned}$$

Using the definitions of $\tilde{Q}_k$, $\tilde{S}_k$ and $K_k = -\tilde{R}_k^{-1}\tilde{S}_k$, the coefficient of x_k simplifies to

$$\begin{aligned}
Q_k + S_k^\top K_k + A_k^\top P_{k+1}(A_k + B_k K_k) &= \tilde{Q}_k + \tilde{S}_k^\top K_k \\
&= \tilde{Q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1}\tilde{S}_k \\
&= P_k
\end{aligned}$$

and the constant part simplifies to $p_k = \tilde{q}_k + \tilde{S}_k^\top \kappa_k$ by an analogous calculation. The remaining KKT conditions hold as well: the terminal condition $\pi_N = Q_N x_N + q_N$ follows

from $P_N = Q_N$ and $p_N = q_N$, and with $\pi_{k+1} = P_{k+1}x_{k+1} + p_{k+1}$ the input condition $R_k u_k + S_k x_k + r_k + B_k^\top \pi_{k+1} = 0$ becomes $\tilde{R}_k u_k + \tilde{S}_k x_k + \tilde{r}_k = 0$, which the optimal policy satisfies by construction. Thus $\pi_k = P_k x_k + p_k = \nabla_{x_k} V_k(x_k)$, which confirms the classical result [3]: *the costate is the gradient of the value function.* This is the link between the KKT perspective and the dynamic programming perspective. The Riccati recursion computes the value function backward in time, which implicitly solves the backward costate dynamics. The optimal policy $u_k^* = K_k x_k + \kappa_k$ then provides the missing coupling that decouples the two-point BVP into two sequential passes.

3.3. The Riccati Recursion. The dynamic programming argument of Section 3.2.2 established that the optimal cost-to-go is quadratic in the state,

$$V_k(x_k) = \tfrac{1}{2} x_k^\top P_k x_k + p_k^\top x_k + \eta_k,$$

and that each backward step minimizes the stage cost plus the downstream cost-to-go over the current input. Writing this minimization out in closed form yields the *Riccati recursion* [6, 7], which solves the unconstrained LQ-OCP in two passes over the horizon: a *backward pass* that propagates the value function and computes the affine feedback law, followed by a *forward pass* that rolls out the optimal trajectory. The backward pass is built on the stage $\mathcal{Q}$-function

$$\mathcal{Q}_k(u_k, x_k) := l_k(u_k, x_k) + V_{k+1}(A_k x_k + B_k u_k + b_k).$$

Its Hessian blocks and gradient are the tilde quantities below, i.e. the stage cost matrices augmented by the propagated value function (P_{k+1}, p_{k+1}).

Backward pass ($k = N-1, N-2, \ldots, 0$). Initialize at the terminal stage with $P_N = Q_N$, $p_N = q_N$. For each stage, first assemble the $\mathcal{Q}$-function blocks

$$\begin{aligned}
\tilde{Q}_k &= Q_k + A_k^\top P_{k+1} A_k \\
\tilde{S}_k &= S_k + B_k^\top P_{k+1} A_k \\
\tilde{R}_k &= R_k + B_k^\top P_{k+1} B_k \\
\tilde{q}_k &= q_k + A_k^\top (P_{k+1} b_k + p_{k+1}) \\
\tilde{r}_k &= r_k + B_k^\top (P_{k+1} b_k + p_{k+1}),
\end{aligned}$$

then minimize $\mathcal{Q}_k$ over u_k to obtain the affine feedback law

$$\begin{aligned}
K_k &= -\tilde{R}_k^{-1} \tilde{S}_k \\
\kappa_k &= -\tilde{R}_k^{-1} \tilde{r}_k,
\end{aligned}$$

and finally update the value function by substituting the minimizer $u_k^* = K_k x_k + \kappa_k$ back into $\mathcal{Q}_k$ (a Schur complement with respect to the $\tilde{R}_k$ block)

$$\begin{aligned}
P_k &= \tilde{Q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1} \tilde{S}_k = \tilde{Q}_k + \tilde{S}_k^\top K_k \\
p_k &= \tilde{q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1} \tilde{r}_k = \tilde{q}_k + \tilde{S}_k^\top \kappa_k.
\end{aligned}$$

The right-hand forms reuse K_k and κ_k, so $\tilde{R}_k$ is factored only once per stage. Since $\tilde{S}_k \in \mathbb{R}^{n_u \times n_x}$ has the same orientation as the cross-weight S_k, the gain $K_k \in \mathbb{R}^{n_u \times n_x}$ maps states to inputs directly.

Forward pass ($k = 0, 1, \ldots, N-1$). Starting from $x_0 = \hat{x}$, apply the feedback law and roll the dynamics forward

$$\begin{aligned}
u_k^* &= K_k x_k + \kappa_k \\
x_{k+1} &= A_k x_k + B_k u_k^* + b_k.
\end{aligned}$$

If the costates are required, they follow from the value-function gradient,

$$\pi_k = \nabla_{x_k} V_k(x_k) = P_k x_k + p_k.$$

Algorithm 1 Classical Riccati recursion

Require: dynamics A_k, B_k, b_k; cost Q_k, R_k, S_k, q_k, r_k with $R_k \succ 0$, $H_k \succeq 0$, $k \in \mathcal{I}_0^{N-1}$; terminal $Q_N \succeq 0$, q_N; initial state $\hat{x}$ (sufficient for $\tilde{R} \succ 0$ in line 6)

Ensure: feedback K_k, κ_k; trajectory x_k, u_k; (optional) costates π_k

Backward pass ▷ propagate the cost-to-go P_k itself, lower triangle

1: $P_N \leftarrow Q_N, \quad p_N \leftarrow q_N$
2: **for** $k = N-1, \dots, 0$ **do**
3: $\quad \begin{bmatrix} \tilde{R} & \tilde{S} \\ \tilde{S}^\top & \tilde{Q} \end{bmatrix} \leftarrow \begin{bmatrix} R_k & S_k \\ S_k^\top & Q_k \end{bmatrix} + \begin{bmatrix} B_k^\top \\ A_k^\top \end{bmatrix} P_{k+1} \begin{bmatrix} B_k & A_k \end{bmatrix}$ ▷ `symm`, `gemm`
4: $\quad w \leftarrow P_{k+1} b_k + p_{k+1}$ ▷ `symv`
5: $\quad \begin{bmatrix} \tilde{r} \\ \tilde{q} \end{bmatrix} \leftarrow \begin{bmatrix} r_k \\ q_k \end{bmatrix} + \begin{bmatrix} B_k^\top \\ A_k^\top \end{bmatrix} w$ ▷ `gemv`
6: $\quad$ factor $\tilde{R} = L_R L_R^\top$ ▷ `potrf`
7: $\quad M \leftarrow \tilde{S}^\top L_R^{-\top}$ ▷ `trsm`
8: $\quad P_k \leftarrow \tilde{Q} - M M^\top$ ▷ `syrk`, lower triangle
9: $\quad \zeta \leftarrow L_R^{-1} \tilde{r}$ ▷ `trsv`
10: $\quad \begin{bmatrix} K_k & \kappa_k \end{bmatrix} \leftarrow -L_R^{-\top} \begin{bmatrix} M^\top & \zeta \end{bmatrix}$ ▷ `trsm`
11: $\quad p_k \leftarrow \tilde{q} - M\zeta$ ▷ `gemv`
12: **end for**

Forward pass

13: $x_0 \leftarrow \hat{x}$
14: **for** $k = 0, \dots, N-1$ **do**
15: $\quad u_k \leftarrow K_k x_k + \kappa_k$
16: $\quad x_{k+1} \leftarrow A_k x_k + B_k u_k + b_k$
17: $\quad$ *(optional)* $\pi_k \leftarrow P_k x_k + p_k$
18: **end for**
19: *(optional)* $\pi_N \leftarrow P_N x_N + p_N$

Classical implementation. Algorithm 1 implements the recursion by propagating the cost-to-go matrix P_k itself [8]. Each stage requires a single Cholesky factorization $\tilde{R}_k = L_R L_R^\top$, from which $M = \tilde{S}_k^\top L_R^{-\top}$ yields both the Schur complement $P_k = \tilde{Q}_k - MM^\top$ and the feedback law. The recursion is well defined as long as $\tilde{R}_k \in \mathbb{S}_{++}^{n_u}$ at every stage, which is guaranteed by the standing assumptions $H_k \succ 0$ for $k < N$ and $Q_N \succ 0$: as shown in Section 3.2.2, $\tilde{H}_k \succeq H_k \succ 0$, so $\tilde{R}_k \succ 0$ and $P_k \succ 0$ at every stage. These assumptions are sufficient, not necessary. Existence and uniqueness only require $R_k \succ 0$, $H_k \succeq 0$ and $Q_N \succeq 0$, under which the classical recursion remains well defined with $P_k \succeq 0$ [6]. These are the requirements stated in Algorithm 1. Per stage, the work is dominated by propagating P_{k+1} through the dynamics, i.e. the dense products $A_k^\top P_{k+1} A_k$, $B_k^\top P_{k+1} A_k$, $B_k^\top P_{k+1} B_k$, together with the factorization of $\tilde{R}_k$ and the Schur-complement product $\tilde{S}_k^\top \tilde{R}_k^{-1} \tilde{S}_k$. Each is $\mathcal{O}\big((n_x + n_u)^3\big)$, so the whole recursion costs

$$\mathcal{O}\big(N(n_x + n_u)^3\big)\,,$$

i.e. *linear* in the horizon length N and cubic in the stage dimension, compared to $\mathcal{O}\big(N^3(n_x + n_u)^3\big)$ for a dense factorization of the full KKT system [8].

Square-root Riccati recursion. Algorithm 1 assembles P_{k+1} explicitly and propagates it through the dynamics at every stage. Propagating a Cholesky factor $L_{P,k}$ instead [8], with $P_k = L_{P,k} L_{P,k}^\top$, is numerically preferable [13]: since $\mathrm{cond}(P_k) = \mathrm{cond}(L_{P,k})^2$, every operation carried out on the factor works at the square root of the condition number of P_k and hence loses only half as many digits to rounding. The derivation rests on the observation that one backward step is entirely a *Gram-matrix* operation. With the stage Hessian H_k and the input-major dynamics $G_k = \begin{bmatrix} B_k & A_k \end{bmatrix}$, the augmented stage

Hessian is H_k plus a congruence of P_{k+1},

$$\tilde{H}_k = H_k + G_k^\top P_{k+1} G_k = H_k + W^\top W, \qquad W := L_{P,k+1}^\top G_k,$$

so substituting $P_{k+1} = L_{P,k+1} L_{P,k+1}^\top$ replaces the congruence $G_k^\top P_{k+1} G_k$ by the Gram matrix $W^\top W$. The matrix P_{k+1} is never assembled, only its factor is read through the dynamics. It remains to extract $L_{P,k}$. The Cholesky factorization of $\tilde{H}_k$ in $[u; x]$ order gives a block lower-triangular factor whose trailing block is precisely the factor of the Schur complement,

$$\tilde{H}_k = \begin{bmatrix} L_R & 0 \\ M & L_{P,k} \end{bmatrix} \begin{bmatrix} L_R^\top & M^\top \\ 0 & L_{P,k}^\top \end{bmatrix}, \qquad L_{P,k} L_{P,k}^\top = \tilde{Q}_k - M M^\top = \tilde{Q}_k - \tilde{S}_k^\top \tilde{R}_k^{-1} \tilde{S}_k = P_k.$$

A single factorization thus realizes both the congruence and the Schur complement that define a Riccati step, and the recursion maps factor to factor, $L_{P,k+1} \mapsto L_{P,k}$, without P ever appearing. This is the discrete-time LQ dual of square-root covariance (Kalman) filtering [12, 13]. The linear terms ride along as a plain vector p_k, updated by a few matrix–vector products.

The factorization above is also where the input-major ordering $[u; x]$ of the stage variables pays off [7]. A Cholesky factorization eliminates the variables in index order, so the trailing block of a lower-triangular factor is always the factor of the Schur complement of the leading block. One Riccati step is precisely the Schur complement of $\tilde{H}_k$ with respect to $\tilde{R}_k$, i.e. the elimination of u_k. With u_k ordered first, a standard lower Cholesky factorization (`potrf`) therefore returns $L_{P,k}$ as its trailing block, ready to be used in the next stage. In the state-major ordering $[x; u]$, the same factorization would eliminate x_k first and return the factor of $\tilde{R}_k - \tilde{S}_k \tilde{Q}_k^{-1} \tilde{S}_k^\top$, which is not the cost-to-go. The upper variant $\tilde{H}_k = U^\top U$ offers no way out, since $U = L^\top$ describes the same elimination order. One would need a reverse factorization $\tilde{H}_k = U U^\top$, which standard LAPACK does not provide, or an explicit permutation at every stage. The classical form is indifferent to the ordering, because it addresses the blocks $\tilde{R}_k$, $\tilde{S}_k$ and $\tilde{Q}_k$ explicitly.

Note that one squaring remains: $\tilde{H}_k = H_k + W^\top W$ is still formed explicitly, so the factorization in line 7 of Algorithm 2 operates on a Gram matrix and concedes part of the digits gained by carrying $L_{P,k}$. This is immaterial as long as $\tilde{H}_k$ is reasonably well conditioned.

Since the full $\tilde{H}_k$ is factored, the square-root form needs $\tilde{H}_k \succ 0$ rather than only $\tilde{R}_k \succ 0$, as well as $Q_N \succ 0$ for the initial factorization. Under the standing assumptions this holds by the argument given for the classical form. Under the weaker assumptions $R_k \succ 0$ and $H_k \succeq 0$, with $Q_N \succ 0$ still required, and given $P_{k+1} \succ 0$, it holds if and only if $\ker(Q_k - S_k^\top R_k^{-1} S_k) \cap \ker(A_k - B_k R_k^{-1} S_k) = \{0\}$. If this holds at every stage, $P_k \succ 0$ follows by induction. For $S_k = 0$ the condition reads $\ker Q_k \cap \ker A_k = \{0\}$, which holds for any rank-deficient $Q_k \succeq 0$ as long as A_k is invertible, as it is for the zero-order-hold discretization of a continuous-time model. Delay states modeled as a shift register are the typical exception. If the condition fails, a regularization $Q_k + \varepsilon I$ restores it. Algorithm 2 states the resulting recursion.

The recursion is fast because its flops are concentrated in a few dense level-3 kernels on small triangular and symmetric operands [10]: one triangular product (`trmm`) to form W, one symmetric rank-n_x update (`syrk`) to build $W^\top W$, one Cholesky factorization (`potrf`) of the result, and a triangular solve with multiple right-hand sides (`trsm`) for the gain. The asymptotic cost is the same as for the classical form, $\mathcal{O}\big(N(n_x + n_u)^3\big)$, but the leading-order flop count is lower: propagating P_{k+1} through the dynamics takes a triangular product (`trmm`) instead of a symmetric one (`symm`), which saves more than the additional factorization of the trailing block costs [8].

Algorithm 2 Square-root Riccati recursion — Cholesky form

Require: dynamics A_k, B_k, b_k; cost Q_k, R_k, S_k, q_k, r_k with $H_k \succ 0$, $k \in \mathcal{I}_0^{N-1}$; terminal $Q_N \succ 0$, q_N; initial state $\hat{x}$ (sufficient for $\tilde{H} \succ 0$ in line 7)

Ensure: feedback K_k, κ_k; trajectory x_k, u_k; (optional) costates π_k

Backward pass ▷ propagate the factor $L_{P,k}$ of the cost-to-go $P_k = L_{P,k} L_{P,k}^\top$

1: factor $Q_N = L_{P,N} L_{P,N}^\top$, $p_N \leftarrow q_N$ ▷ `potrf`

2: **for** $k = N-1, \dots, 0$ **do**

3: $W \leftarrow L_{P,k+1}^\top \begin{bmatrix} B_k & A_k \end{bmatrix}$ ▷ `trmm`, $W \in \mathbb{R}^{n_x \times (n_u + n_x)}$

4: $w \leftarrow L_{P,k+1} \Big(L_{P,k+1}^\top b_k \Big) + p_{k+1}$ ▷ `trmv`, `trmv`, `axpy`

5: $\tilde{H} \leftarrow \begin{bmatrix} R_k & S_k \\ S_k^\top & Q_k \end{bmatrix} + W^\top W$ ▷ `syrk`, lower triangle

6: $\begin{bmatrix} \tilde{r} \\ \tilde{q} \end{bmatrix} \leftarrow \begin{bmatrix} r_k \\ q_k \end{bmatrix} + \begin{bmatrix} B_k^\top \\ A_k^\top \end{bmatrix} w$ ▷ `gemv`

7: factor $\tilde{H} = \begin{bmatrix} L_R & 0 \\ M & L_{P,k} \end{bmatrix} \begin{bmatrix} L_R^\top & M^\top \\ 0 & L_{P,k}^\top \end{bmatrix}$ ▷ `potrf`, $[u; x]$ order

8: $\zeta \leftarrow L_R^{-1} \tilde{r}$ ▷ `trsv`

9: $\begin{bmatrix} K_k & \kappa_k \end{bmatrix} \leftarrow -L_R^{-\top} \begin{bmatrix} M^\top & \zeta \end{bmatrix}$ ▷ `trsm`

10: $p_k \leftarrow \tilde{q} - M\zeta$ ▷ `gemv`

11: **end for**

Forward pass

12: $x_0 \leftarrow \hat{x}$

13: **for** $k = 0, \dots, N-1$ **do**

14: $u_k \leftarrow K_k x_k + \kappa_k$

15: $x_{k+1} \leftarrow A_k x_k + B_k u_k + b_k$

16: *(optional)* $\pi_k \leftarrow L_{P,k} \Big(L_{P,k}^\top x_k \Big) + p_k$

17: **end for**

18: *(optional)* $\pi_N \leftarrow L_{P,N} \big(L_{P,N}^\top x_N \big) + p_N$

References

[1] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[2] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume 1. Athena Scientific, Belmont, MA, 1995.

[3] Arthur E. Bryson and Yu-Chi Ho. *Applied Optimal Control: Optimization, Estimation, and Control*. Hemisphere Publishing Corporation / Halsted Press (John Wiley & Sons), Washington, D.C. / New York, revised edition, 1975. ISBN 978-0470114810.

[4] Alexander Domahidi, Aldo U Zgraggen, Melanie N Zeilinger, Manfred Morari, and Colin N Jones. Efficient interior point methods for multistage problems arising in receding horizon control. In *2012 IEEE 51st IEEE conference on decision and control (CDC)*, pages 668–674. IEEE, 2012.

[5] Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: A parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6(4):327–363, 2014.

[6] Gianluca Frison. *Algorithms and Methods for High-Performance Model Predictive Control*. PhD thesis, Technical University of Denmark, 2015.

[7] Gianluca Frison and Moritz Diehl. HPIPM: a high-performance quadratic programming framework for model predictive control. *IFAC-PapersOnLine*, 53(2):6563–6569, 2020.

[8] Gianluca Frison and John Bagterp Jørgensen. Efficient implementation of the riccati recursion for solving linear-quadratic control problems. In *2013 IEEE International Conference on Control Applications (CCA)*, pages 1117–1122. IEEE, 2013.

[9] Gianluca Frison and John Bagterp Jørgensen. A fast condensing method for solution of linear-quadratic control problems. In *52nd IEEE Conference on Decision and Control*, pages 7715–7720. IEEE, 2013.

[10] Gianluca Frison, Dimitris Kouzoupis, Tommaso Sartor, Andrea Zanelli, and Moritz Diehl. BLASFEO: Basic linear algebra subroutines for embedded optimization. *ACM Transactions on Mathematical Software*, 44(4):42:1–42:30, 2018.

[11] Rudolf Emil Kalman. Contributions to the theory of optimal control. *Boletín de la Sociedad Matemática Mexicana*, 5(2):102–119, 1960.

[12] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[13] Paul Kaminski, Arthur Bryson, and Stanley Schmidt. Discrete square root filtering: A survey of current techniques. *IEEE Transactions on Automatic Control*, 16(6): 727–736, 1971.

[14] Yang Wang and Stephen Boyd. Fast model predictive control using online optimization. *IEEE Transactions on Control Systems Technology*, 18(2):267–278, 2010.

[15] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3): 279–292, 1992.